

Digital Microelectronics

Näf Gian Marco

6. Januar 2026

1 How to Design a System on Chip

1.1 Motivation und Einordnung

Moderne digitale Systeme sind durch steigende Komplexität, hohe Heterogenität und starken Zeitdruck geprägt. Klassische FPGA- oder HDL-basierte Schaltungsentwicklung skaliert nicht mehr. Aktuelle Designs sind vollständige Systeme mit Prozessoren, Speicherhierarchien, Peripherie und applikationsspezifischer Logik auf einem einzigen Chip.

1.2 Begriff und Eigenschaften eines SoC

Ein System on Chip integriert Mikroprozessor(en), Speicher, Standardperipherie und anwendungsspezifische IP-Blöcke auf einem Silizium-Die. Ziel ist eine hohe Integrationsdichte bei reduzierten Entwicklungs- und Systemkosten sowie erhöhter Flexibilität durch Software-Updates und rekonfigurierbare Hardware.

FPGA-basierte SoCs kombinieren feste Hard-IP (Processing System) mit rekonfigurierbarer Logik (Programmable Logic). Dies erlaubt Hardware/Software-Co-Design und iterative Systementwicklung.

1.3 Zynq-7000 Architekturüberblick

Die Zynq-7000 Plattform besteht aus zwei Hauptdomänen:

- **Processing System (PS):** Dual ARM Cortex-A9, Caches, Speichercontroller, feste Peripherie, Hard-IP.
- **Programmable Logic (PL):** 7-Series FPGA-Logik mit CLBs, Block RAM, DSP48E1-Slices, I/O-Blöcken und Clock-Ressourcen.

PS und PL sind eng gekoppelt und kommunizieren über dedizierte AXI-Interfaces. Ergänzt wird das System durch analoge Funktionen wie XADC (ADC, Sensorik).

1.4 Ressourcen der Programmable Logic

Die PL basiert auf einer feingranularen Logikstruktur:

- **CLB (Configurable Logic Block):** Besteht aus zwei Slices.
- **Slice:** Enthält LUTs, Flip-Flops, Carry-Chains und Multiplexer.
- **LUTs:** 6-Eingangs-LUTs mit Dual-Output, konfigurierbar als zwei 5-Eingangs-LUTs.
- **Block RAM:** Dual-Port, vollständig synchron, optional gepipelined.
- **DSP48E1:** Multiplizierer, Pre-Adder, Post-Adder, SIMD-fähig.

Diese Ressourcen ermöglichen hochparallele, datenpfadorientierte Implementierungen.

1.5 I/O- und Analogressourcen

Die 7-Series unterstützen High-Range- und High-Performance-I/Os mit LVDS, SERDES, Delay-Elementen und differenziellen Signalen. Ergänzend bietet das XADC-System integrierte ADCs und On-Chip-Sensorik für Mixed-Signal-Anwendungen.

1.6 SoC-Design-Herausforderungen

Zentrale Herausforderungen moderner SoC-Entwicklung:

- Systemkomplexität und Parallelität
- Hardware/Software-Partitionierung
- Mehrere Clock-Domänen und Timing Closure
- IP-Integration und Wiederverwendbarkeit
- Team-basierte Entwicklungsflüsse

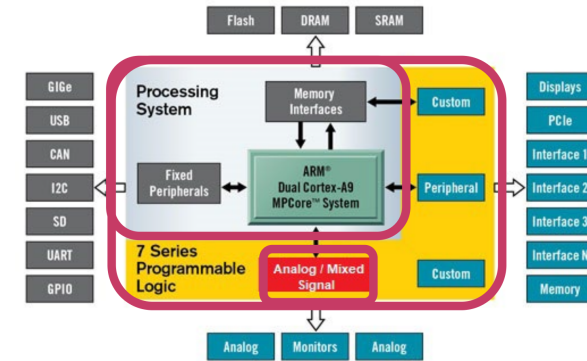


Abbildung 1: Zynq-7000 Gesamtarchitektur (PS/PL).

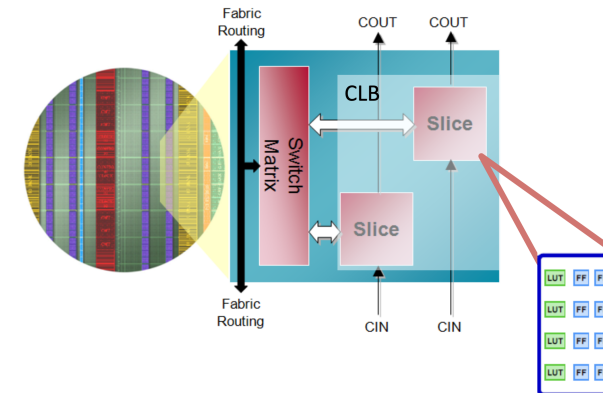


Abbildung 2: CLB- und Slice-Struktur der PL.

FPGA-Designs nähern sich methodisch ASIC-Plattformdesigns an.

1.7 Design Flow und Tool-Unterstützung

Der klassische RTL-zentrierte FPGA-Flow wird durch einen systemorientierten, IP-basierten Ansatz ergänzt. Kernelemente:

- Abstraktionserhöhung
- HW/SW-Co-Design
- IP-Integration
- Einheitliches Constraint-System (XDC)
- Durchgängige Reports und Checkpoints

Vivado dient als zentrale Entwicklungsumgebung für Systemdesign, Implementierung, Analyse und Debugging.

2 SoC Constraints

2.1 Rolle von Constraints im SoC-Design

HDL beschreibt primär Funktionalität, nicht physikalische oder zeitliche Eigenschaften der Hardware. Moderne SoC-Designs sind ohne explizite Constraints nicht korrekt implementierbar.

Merksatz (Prüfung): Constraints liefern dem Tool alle Informationen zu Pins, Spannungen und Zeiten ,

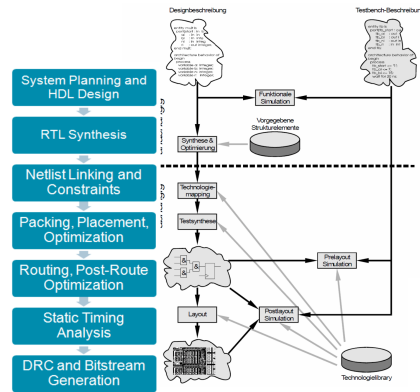


Abbildung 3: Traditioneller RTL/FPGA-Flow vs. System-Level/IP-Flow.

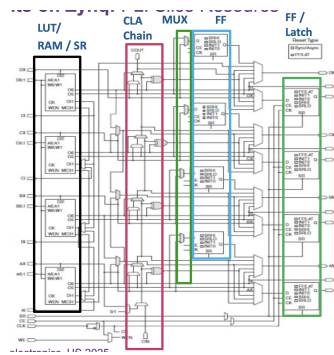


Abbildung 4: Entwicklung von PLD über FPGA zu SoC.

die im HDL fehlen.

Constraints steuern Synthese, Platzierung, Routing und Timing-Optimierung.

2.2 Arten von Constraints

Constraints lassen sich in drei Hauptkategorien einteilen:

- **Physikalische Constraints:** Pin-Zuordnung, I/O-Standards, Nutzung spezifischer FPGA-Ressourcen, Platzierung von Zellen
- **Timing Constraints:** Clock-Definition, Input- und Output-Delays, Timing-Ausnahmen
- **Konfigurations-Constraints:** Bitstream-Optionen, Konfigurationsmodi

Prüfungsregel: Timing - Constraints wirken bereits in der Synthese, physikalische Constraints primär in der Implementierung.

2.3 XDC-Format und Constraint-Organisation

Vivado verwendet XDC-Dateien auf Basis von Synopsys Design Constraints (SDC). Constraints folgen Tcl-Syntax und werden **sequentiell** interpretiert.

Empfohlene Struktur:

- Timing Assertions (Clocks, I/O-Delays)
- Timing Exceptions (False Paths, Multicycle Paths)
- Physikalische Constraints (Pins, I/O-Standards)

Merksatz: Timing zuerst, Pins zuletzt.

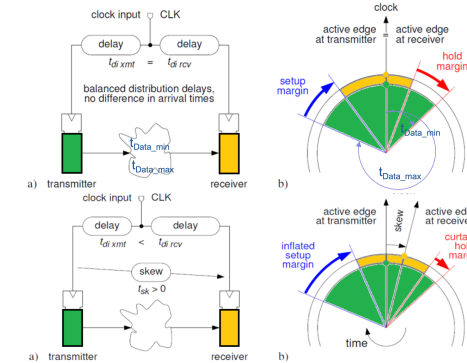


Abbildung 5: Timing-Nichtidealitäten: Latency, Skew und Jitter.

2.4 Physikalische Constraints

Physikalische Constraints definieren die Abbildung logischer Ports auf physische Pins sowie elektrische Eigenschaften:

- Pin-Zuweisung (PACKAGE_PIN)
- I/O-Standard (IOSTANDARD)
- Pull-Ups, Pull-Downs

Vorgehensschema (Prüfung)

1. Signalrichtung bestimmen
2. Spannung bestimmen → IOSTANDARD
3. Pin zuordnen → PACKAGE_PIN
4. Reset? → PULLUP true

XDC-Code-Schablone

```
1 set_property IOSTANDARD LVCMOS33 [get_ports rst]
2 set_property PULLUP true [get_ports rst]
3 set_property PACKAGE_PIN V13 [get_ports rst]
```

In Sonderfällen können explizite Platzierungsconstraints (LOC, BEL) verwendet werden.

2.5 Timing-Grundlagen und Begriffe

Timing-Verifikation erfolgt mittels Static Timing Analysis (STA).

Zentrale Begriffe:

- Clock Latency
- Clock Skew
- Clock Jitter
- Slack

Prüfungsregel: Timing ist korrekt, wenn Setup- und Hold-Bedingungen erfüllt sind und der Slack ≥ 0 ist.

2.6 Static Timing Analysis (STA)

STA analysiert alle relevanten Minimal- und Maximalpfade ohne Simulation:

- Clock-to-Clock-Pfade
- Setup-, Hold- und Pulse-Width-Bedingungen

Wichtige Kennzahlen:

- Worst Negative Slack (WNS)
- Total Negative Slack (TNS)

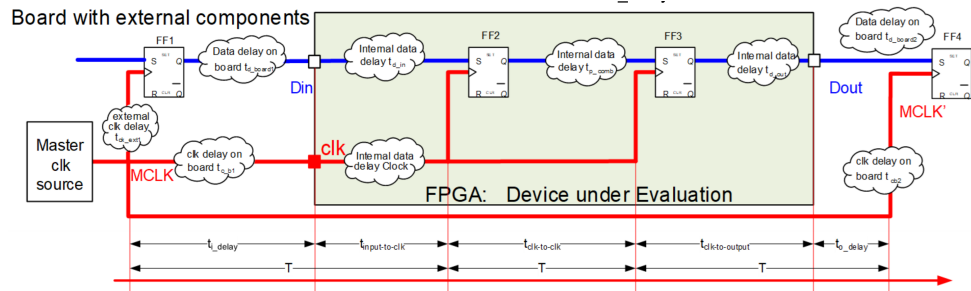


Abbildung 6: Input- und Output-Delay-Pfade mit externen Komponenten.

2.7 Clock-Definition

Jede relevante Clock muss explizit definiert werden.

Formel

$$T_{clk} = \frac{1}{f}$$

Code-Schablone

```
1 create_clock -name clk -period 10 [get_ports clk]
```

2.8 Input-Delay-Constraints

Externe Signalankunftszeiten müssen spezifiziert werden.

Prüfungsformel

$$t_{input_delay} = t_{data} - t_{clk}$$

Code-Schablone

```
1 set_input_delay -clock clk 3 [get_ports data_in]
```

2.9 Output-Delay-Constraints

Output-Delays definieren erforderliche externe Setup- und Hold-Zeiten.

Code-Schablone

```
2 set_output_delay -clock clk 5 [get_ports data_out]
```

2.10 Wirkung von Constraints

Constraints beeinflussen Platzierung, Routing und Optimierung direkt.

Abschluss-Merkatz (Prüfung): Ohne korrekte Constraints kein valides Timing und kein reproduzierbares Design.

3 System Level VHDL und IP-Wiederverwendung

3.1 Zielsetzung von System Level VHDL

Moderne SoC-Entwicklung erfordert skalierbare, parametrisierbare und wiederverwendbare Hardwarebeschreibungen. System Level VHDL adressiert Komplexität, Heterogenität und Time-to-Market durch Hierarchie, Modularisierung und Abstraktion.

Merksatz (Prüfung): System Level VHDL zielt auf Wiederverwendung und Skalierung, nicht auf Gate-Optimierung.

Ziel ist die Wiederverwendung bewährter IP-Blöcke statt monolithischer, hardcodierter Designs.

3.2 Grundprinzipien der Wiederverwendung

Wiederverwendbarer VHDL-Code folgt klaren Regeln:

- Keine fest kodierten Parameter
- Klare Schnittstellen
- Parametrisierung über Generics
- Zentrale Definition von Konstanten und Typen
- Saubere Dokumentation und Kommentare

Prüfungsregel: Projektweit konstant $\Rightarrow$ Package, Variabel pro Instanz $\Rightarrow$ Generic.

Code wird typischerweise von anderen Entwicklern genutzt als vom ursprünglichen Autor. Lesbarkeit ist ein funktionales Kriterium.

3.3 Alias, Konstanten und Generics

Aliases verbessern die Lesbarkeit, insbesondere bei Vektorslices.

Alias – Code-Schablone

```
1 alias opcode : std_logic_vector(3 downto 0) is instr(15 downto 12);
```

Konstanten definieren unveränderliche Entwurfsparameter und erhöhen Konsistenz.

Konstante – Code-Schablone

```
3 constant DATA_WIDTH : integer := 16;
```

Generics ermöglichen die Übergabe statischer Parameter von außen in eine Entity.

Generic – Code-Schablone

```
1 entity alu is
2   generic ( WIDTH : integer := 16 );
3   port (
4     a, b : in  std_logic_vector(WIDTH-1 downto 0);
5     y    : out std_logic_vector(WIDTH-1 downto 0)
6   );
7 end entity;
```

Merksatz: Generics werden bei der Instanziierung festgelegt und sind zur Laufzeit konstant.

3.4 Generate-Konstrukte

Generate-Anweisungen erlauben die strukturierte Erzeugung mehrfacher Instanzen. In Kombination mit Generics entstehen skalierbare Hardwarestrukturen.

Typische Einsatzfälle:

- Replikation identischer Verarbeitungseinheiten
- Parametrisierbare Breiten und Tiefen
- Strukturierte Arrays von Modulen

Prüfungsregel: Generate ist elaborationszeitlich wirksam und erzeugt echte Hardware.

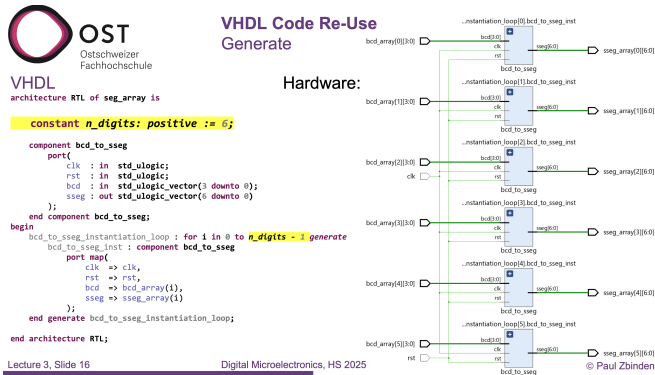


Abbildung 7: Generate-Struktur und resultierende Hardware.

Generate – Code-Schablone

```

4 gen_loop : for i in 0 to N-1 generate
5   u : entity work.reg
6   port map (clk => clk, d => din(i), q => dout(i));
7 end generate;

```

3.5 Funktionen in VHDL

Funktionen sind nebenläufig nutzbare Unterprogramme mit ausschließlich Eingangsparametern und genau einem Rückgabewert. Sie werden wie Ausdrücke verwendet und dienen der strukturellen Abstraktion.

Eigenschaften:

- Keine Signalzuweisungen
- Keine Wait-Statements
- Verwendung von Variablen
- Rekursion erlaubt

Funktion – Code-Schablone

```

1 function max(a,b : integer) return integer is
2 begin
3   if a > b then return a; else return b; end if;
4 end function;

```

Funktionen eignen sich besonders für kombinatorische Logik und arithmetische Operationen.

3.6 Prozeduren in VHDL

Prozeduren können mehrere Ein- und Ausgänge besitzen. Sie sind mächtiger als Funktionen, jedoch eingeschränkt synthesefähig. Hauptanwendungsgebiet sind Testbenches und Verhaltensmodelle.

Prüfungsregel: Prozeduren in Synthese sind toolabhängig und kritisch zu prüfen.

3.7 Arrays und Records

Arrays modellieren Sammlungen gleichartiger Elemente. Sie können eindimensional oder mehrdimensional sein und sowohl natürliche als auch enumerierte Indizes verwenden.

Array – Code-Schablone

```

8 type vec_t is array (0 to 7) of std_logic_vector(15 downto 0);
9 signal v : vec_t;

```

Records gruppieren Elemente unterschiedlicher Typen zu logischen Strukturen. Sie eignen sich für Busse, Transaktionen und strukturierte Datensätze im Systementwurf.

Arrays may be multi-dimensional:

Dimension	1D x 1D (Integer)	1D x 1D (bit_vector)	2D (bits)			
Slice 1	3	"0011"	'0'	'0'	'1'	'1'
Slice 2	9	"1001"	'1'	'0'	'0'	'1'
Slice 3	13	"1101"	'1'	'1'	'0'	'1'
VHDL	a)	b)	c)			

- a) type OneD_x_OneD_Int is array (1 to 3) of integer range 0 to 15;
b) type OneD_x_oneD_vector is array (1 to 3) of bit_vector(1 to 4);
c) type TwoD_x_oneD_Bit is array(1 to 3, 1 to 4) of bit;

Abbildung 8: Array- und Record-Beispiele mit Hardwareabbildung.

Record – Code-Schablone

```

1 type bus_t is record
2   addr : std_logic_vector(15 downto 0);
3   data : std_logic_vector(31 downto 0);
4   we : std_logic;
5 end record;

```

3.8 Packages

Packages sind zentrale Container für projektweite Definitionen:

- Konstanten
- Typen
- Komponenten
- Funktionen und Prozeduren

Sie fördern Konsistenz, Wiederverwendung und Wartbarkeit. Packages werden kompiliert in Libraries abgelegt und über use-Anweisungen eingebunden.

Package – Minimal-Schablone

```

10 package defs is
11   constant WIDTH : integer := 16;
12   function inc(x : integer) return integer;
13 end package;
14
15 package body defs is
16   function inc(x : integer) return integer is
17   begin
18     return x + 1;
19   end function;
20 end package body;

```

Package verwenden

```

1 use work.defs.all;

```

Typische Anwendung: Projekt-Settings, mathematische Hilfsfunktionen, Protokollparameter.

3.9 System-Level-Designwirkung

System Level VHDL verschiebt den Fokus von signalorientierter Beschreibung hin zu strukturierter Systemarchitektur. Es bildet die Grundlage für IP-basierte SoC-Designs und saubere HW/SW-Partitionierung.

Abschluss-Merkatz: Gute Wiederverwendung entsteht durch klare Schnittstellen, Generics und Packages.

Signed / Unsigned

Unsigned $uQn.m$

It's a question of how to interpretation a bit string:

signed => sQn.m / unsigned => uQn.m

$$z = \sum_{k=-m}^{n-1} a_k \cdot B^k \quad \text{Range: } uQn.m = [0 \dots 2^n - 2^{-m}]$$

Example: uQ2.6 bit string "10110100" =

$$1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 0 \cdot 2^{-3} + 1 \cdot 2^{-4} = 2.8125$$



Signed Integer $sQn.m$

$$z = -(2^{n-1}) \cdot a_{n-1} + \sum_{k=-m}^{n-2} a_k \cdot B^k \quad \text{Range: } sQn.m = [-2^{n-1} \dots 2^{n-1} - 2^{-m}]$$

Example: sQ2.6 bit string "10110100" =

$$-1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 0 \cdot 2^{-3} + 1 \cdot 2^{-4} = -1.1875$$

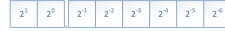


Abbildung 9: Qn.m-Darstellung; signed vs. unsigned, Wertebereich und Auflösung.

4 Fixed Point Arithmetic

4.1 Motivation und Problemstellung

Reale Zahlen treten in digitalen Systemen häufig auf, können jedoch in Hardware nicht direkt verarbeitet werden. Fixed-Point-Arithmetik ermöglicht eine effiziente Approximation reeller Werte mit endlicher Wortlänge.

Merksatz (Prüfung): Bei Fixed Point ist der Entwickler verantwortlich für Skalierung, Wortlänge und Überlauf.

4.2 Ganzzahl- und Zweierkomplement-Darstellung

Binäre Ganzzahlen werden als unsigned oder signed (Zweierkomplement) interpretiert.

Für n Bit gilt:

- Unsigned: $[0, 2^n - 1]$
- Signed: $[-2^{n-1}, 2^{n-1} - 1]$

Prüfungsregel: Das Bitmuster ist identisch, nur der Typ bestimmt die Interpretation.

4.3 Fixed-Point-Zahlendarstellung (Qn.m)

Fixed-Point-Zahlen besitzen einen impliziten Dezimalpunkt. n Bits repräsentieren den ganzzahligen Anteil, m Bits den fractional Anteil.

Notation:

- uQn.m: unsigned
- sQn.m: signed

Auflösung und Wertebereich

$$\Delta = 2^{-m}$$

- $uQn.m = [0, 2^n - 2^{-m}]$
- $sQn.m = [-2^{n-1}, 2^{n-1} - 2^{-m}]$

4.4 Skalierung und Typkonvertierung

Reale Zahlen werden durch Skalierung in Fixed Point überführt.

Prüfungsformeln

$$\text{Fixed} = \lfloor \text{Real} \cdot 2^m \rfloor$$

$$\text{Real} = \text{Fixed} \cdot 2^{-m}$$

Prüfungsregel: Ohne explizite Typkonvertierung keine Arithmetik in VHDL.

Size in Detail

How about the sign bit?

VHDL without fixed_pkg

VHDL with fixed_pkg

Sum: Resize must handle sign bit correctly:

wrong: 1011 => 001011

correct: 1011 => 111011

```
sum <= (A(A'left) & A) + (B(B'left) & B);
```

```
sum <= (resize(A,sum'range)) + (resize(B,sum'range));
```

Guarding and alignment done automatically

Example:

```
-- fixed_pkg
signal sMinuend: sfixed(2 downto -3) := to_sfixed(MINUEND, 2, -3); -- Q3.3
signal sSubtrahend: sfixed(0 downto -4) := to_sfixed(SUBTRAHEND, 0, -4); -- Q1.4
difference <= sMinuend - sSubtrahend;
```

Abbildung 10: Addition: Alignment und Guard Bit zur Überlaufabsicherung.

4.5 fixed_pkg vs. klassische Umsetzung

Die Bibliothek fixed_pkg (VHDL-2008) bietet:

- Automatisches Alignment
- Guard Bits
- Kontrollierte Rundung und Sättigung

Prüfungsfokus: Prüfungen verlangen meist die klassische Umsetzung ohne fixed_pkg.

4.6 Addition und Subtraktion

Formatregel (Prüfung)

$$Q(n_1.m_1) + Q(n_2.m_2) = Q(\max(n_1, n_2) + 1). \max(m_1, m_2)$$

Das zusätzliche Bit ist ein Guard Bit.

Vorgehen ohne fixed_pkg

1. Fractional Bits angleichen
2. Vorzeichenerweiterung
3. Addition
4. Optional: Abschneiden

Additions-Code-Schablone

```
21 A_ext <= signed(A) sll 1;
22 B_ext <= signed(B);
23 Y_ext <= A_ext + B_ext;
```

Prüfungsregel: Addition scheitert meist an fehlendem Guard Bit.

4.7 Multiplikation

Formatregel

$$Q(n_1.m_1) \cdot Q(n_2.m_2) = Q(n_1 + n_2). (m_1 + m_2)$$

Bei signed × signed entsteht ein redundantes Vorzeichenbit:

$$Q(n_1 + n_2 - 1). (m_1 + m_2)$$

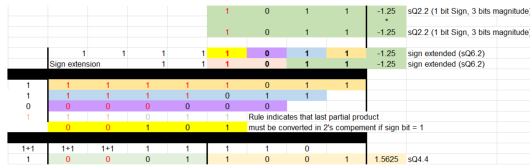
Multiplikations-Code-Schablone

```
1 P <= signed(A) * signed(B);
```

Prüfungsregel: Nach der Multiplikation sind fast immer zu viele Bits vorhanden.

Special Case: $sQ \cdot sQ$
Sign Bit in Product

$Qn1.m1 \cdot Qn2.m2 = Q(n1+n2-1).(m1+m2)$
MSB of the product is a redundant sign bit, because there is a sign bit in both the multiplicand and the multiplier.



Exception: Corner case
1 bit sign, 2 bits magn.

lowest negative number yields overflow: (Example 3 bit)
 $-4 \cdot -4 = +16 \Rightarrow (0b100) \times (0b100) = (0b010000)$

Abbildung 11: Multiplikation: resultierendes Format und redundantes Vorzeichenbit.

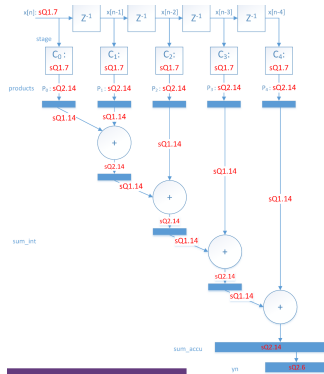


Abbildung 12: FIR-Filter Blockdiagramm mit Q-Formaten für Produkte und Akkumulation.

4.8 Überlauf, Rundung und Sättigung

Mögliche Effekte:

- Wrap-Around (Überlauf)
- Sättigung
- Trunkierung
- Rundung

Prüfungsfokus: Trunkierung und Rundung unterscheiden können.

4.9 FIR-Filter als Anwendungsbeispiel

Ein FIR-Filter besteht aus:

- Verzögerungskette (z^{-1})
- Multiplikationen
- Akkumulation

Wortlängenstrategie (Prüfung):

- Produkt: redundantes Vorzeichenbit entfernen
- Zwischensumme: Guard Bits meist vernachlässigbar
- Endsumme: Guard Bit beibehalten

4.10 Systematische Wortlängenplanung

Effiziente Fixed-Point-Systeme erfordern:

- Analyse der Wertebereiche

- Konsistente Q-Formate
- Kontrollierte Guard Bits

Abschluss-Merkatz: Fixed Point ist kontrollierter Fehler. Gute Planung schlägt hohe Wortlänge.

5 Custom IP Blocks: Memory

5.1 Einordnung und Ziel

Speicher sind zentrale Bausteine digitaler Systeme und dominieren häufig Fläche, Energieverbrauch und Performance. Auf FPGAs existieren keine dedizierten ROM-Strukturen. RAM und ROM werden aus vorhandenen Hardware-Ressourcen abgebildet.

Merkatz (Prüfung): RAM und ROM unterscheiden sich in VHDL primär durch das Zugriffsverhalten, nicht durch die Struktur.

Ziel ist die parametrisierbare Beschreibung, Initialisierung und Wiederverwendung von Speicherstrukturen als IP-Blöcke.

5.2 Speichertypen

Grundsätzlich wird zwischen RAM und ROM unterschieden:

- **RAM:** Lesen und Schreiben zur Laufzeit
- **ROM:** Nur Lesen, konstante Inhalte

Technologisch:

- **SRAM:** Statisch, kein Refresh
- **DRAM:** Dynamisch, Refresh erforderlich

FPGA-spezifisch:

- **Distributed RAM** (LUT-basiert)
- **Block RAM** (dedizierte Hard-Macros)

5.3 Speicherorganisation

Logisch wird ein Speicher beschrieben durch:

- **Breite:** Bits pro Wort
- **Tiefe:** Anzahl Wörter

Prüfungsformeln

$$\text{Tiefe} = 2^{\text{ADDR_WIDTH}}$$

$$\text{Gesamtbits} = \text{Breite} \cdot \text{Tiefe}$$

Prüfungsregel: ADDR _ WIDTH bestimmt die Anzahl Adressen, nicht die Wortbreite.

5.4 Speicherports und Zugriffsarten

Unterstützte Port-Typen:

- **Single Port RAM**
- **Simple Dual Port RAM**
- **True Dual Port RAM**

Unterschiede (Prüfung)

- **Single Port:** Lesen oder Schreiben
- **Simple DP:** ein Schreib-, ein Leseport
- **True DP:** zwei vollwertige Ports

5.5 Zugriffsmodi bei Schreibkollisionen

Bei gleichzeitigem Zugriff auf dieselbe Adresse:

- **Write First**
- **Read First**

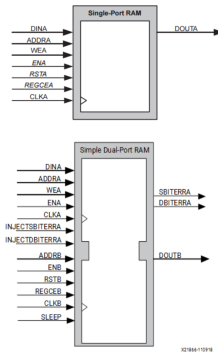


Abbildung 13: RAM-Organisation und Implementierung als Block RAM oder Distributed RAM.

- No Change

Prüfungsregel: Das Verhalten ist nicht implizit, sondern muss spezifiziert werden.

5.6 Distributed RAM

Distributed RAM nutzt LUTs (SLICEMs):

- Schreiben synchron
- Lesen asynchron
- Kleine, schnelle Speicher

Prüfungsregel: A synchron es Lesen → Distributed RAM.

5.7 Block RAM

Block RAM besteht aus dedizierten Hard-Macros:

- 36 kb physisch, ca. 32 kb nutzbar
- Synchrones Lesen und Schreiben
- Unterstützt ECC und FIFO

Prüfungsregel: Synchrones Lesen → Block RAM.

5.8 VHDL-Modellierung von RAM

Grundregeln

- Breite und Tiefe über Generics
- Eindimensionales Array
- Ein Prozess pro Port

Single Port RAM – Code-Schablone

```

24 type ram_t is array (0 to DEPTH-1) of std_logic_vector(W-1 downto 0);
25 signal ram : ram_t;
26
27 process(clk)
28 begin
29     if rising_edge(clk) then
30         if we='1' then
31             ram(addr) <= din;
32         end if;
33         dout <= ram(addr);
34     end if;
35 end process;

```

True Dual Port RAM – Hinweis

True Dual Port RAM wird häufig besser über dedizierte Block-RAM-IP realisiert. Eine VHDL-Modellierung mit `shared variable` ist simulatorfreundlich, aber kritisch.

5.9 Synthesis-Attribute

Die Implementierung kann gezielt beeinflusst werden:

```

36 attribute ram_style : string;
37 attribute ram_style of ram : signal is "block";

```

Alternativ:

```

38 attribute ram_style of ram : signal is "distributed";

```

Merksatz (Prüfung): Attribute sind Hinweise an das Tool, keine funktionalen Anweisungen.

5.10 ROM-Implementierung

ROM unterscheidet sich von RAM nur durch konstante Inhalte.

ROM – Code-Schablone

```

39 type rom_t is array (0 to 15) of std_logic_vector(7 downto 0);
40 constant rom : rom_t := (
41     0 => x"12",
42     1 => x"34",
43     others => x"00"
44 );

```

5.11 Speicherinitialisierung

Möglichkeiten:

- Initialisierung bei Deklaration
- Datei-basierte Initialisierung (.coe, .mif)

Prüfungsregel: ROM ohne Initialisierung ist inhaltlich wertlos.

5.12 Memory Generator IP

Xilinx stellt parametrische Speicher-IP bereit:

- Frei wählbare Breite und Tiefe
- Konfigurierbare Zugriffsmodi
- Native oder AXI4-Schnittstelle

5.13 IP-Wiederverwendung und Integration

IP-Blöcke sind parametrisierte, wiederverwendbare Einheiten.

- Hard IP
- Firm IP
- Soft IP

Integration:

- IP Integrator
- RTL-Instantiation

5.14 IP Lifecycle

IP-Blöcke unterliegen Versions- und Lebenszyklen:

- Pre-Production
- Production
- Superseded
- Discontinued

Abschluss-Merksatz: Gute Speicher-IP ist parametrisch, synchron und eindeutig spezifiziert.

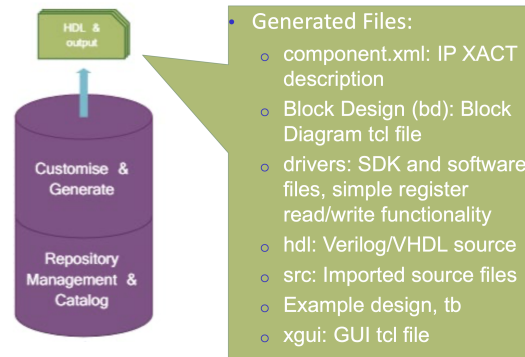


Abbildung 14: ARM-Prototyp und Design-Workflow im IP Integrator.

Point-to-point

Star

Bus

Various combinations

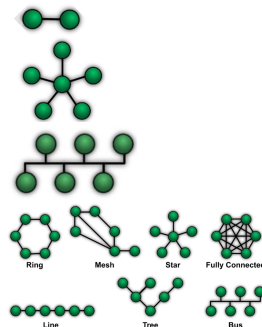


Abbildung 15: Klassifikation serieller Schnittstellen nach Topologie und Datenorganisation.

6 Serial Communication Interfaces

6.1 Systematische Einordnung

Serielle Kommunikationsschnittstellen verbinden digitale Systeme auf Chip-, Board- oder Systemebene. Sie lassen sich entlang mehrerer Dimensionen klassifizieren:

- **Topologie:** Point-to-Point, Bus, Stern, Daisy-Chain
- **Datenorganisation:** Seriell vs. Parallel
- **Zeitorganisation:** Synchron vs. Asynchron
- **Physikalische Umsetzung:** Single-Ended vs. Differenziell

Prüfungsregel: Serielle Kommunikation spart Pins, erhöht aber die Logikkomplexität.

Die Vorlesung behandelt ausschließlich Physical Layer und Data Link Layer.

6.2 Logik-zu-Physik-Übersetzung

VHDL-Signale sind rein logisch und müssen über I/O-Pads physikalisch umgesetzt werden.

Relevante Parameter:

- I/O-Standard
- Single-Ended oder differenziell
- Slew Rate und Drive Strength
- Pull-Up, Pull-Down, Keeper

Merksatz (Prüfung): Pads bestimmen das elektrische Verhalten, nicht das Protokoll.

Hardware

LVDS (Low Voltage Digital Signalling)

- Current – mode instead of voltage – mode
- Low signal swing of only $\pm 350\text{mV}$
- Differential transmission reduces common mode noise

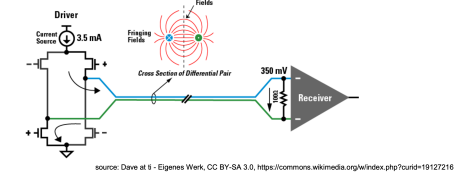


Abbildung 16: LVDS-Prinzip und Umsetzung über FPGA-I/O-Pads.

6.3 Spannungs- und Strommodus

Voltage Mode:

- Logik über Spannungspegel
- Einfach, aber hoher Energieverbrauch

Current Mode:

- Logik über Stromrichtung
- Kleiner Spannungs-Hub
- Besser für hohe Datenraten

Prüfungsregel: Hohe Datenrate $\rightarrow$ Current Mode.

6.4 LVDS

LVDS ist eine differentielle Strommodus-Signalisierung.

Kenndaten:

- Strom: $\pm 3.5\text{ mA}$
- Terminierung: $\approx 100\ \Omega$
- Signallhub: $\approx \pm 350\text{ mV}$

Vorteile:

- Hohe Geschwindigkeit
- Geringe EMV
- Hohe Störfestigkeit

LVDS wird über Constraints konfiguriert und nutzt spezielle Buffer.

6.5 Serialisierung und Deserialisierung

Serielle Schnittstellen benötigen SERDES-Strukturen:

- Parallel-In / Serial-Out (PISO)
- Serial-In / Parallel-Out (SIPO)

Grundbaustein ist ein Schieberegister mit FSM-Steuerung.

Schieberegister – Code-Schablone

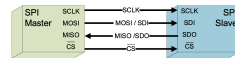
```

45 if rising_edge(clk) then
46   if load='1' then
47     shreg <= data_in;
48   elsif shift='1' then
49     shreg <= shreg(W-2 downto 0) & '0';
50   end if;
51 end if;

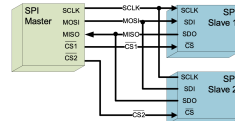
```

Serial Peripheral Interface

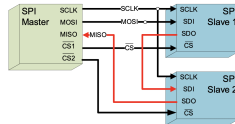
- Single Point – to – Point



- Multi – subnode SPI configuration (star with individual CS – lines)



- Daisy-Chain SPI configuration (one shared CS - line)



Digital Microelectronics, HS 2025

Abbildung 17: SPI-Topologien, Signale und Master-Slave-Prinzip.

6.6 SPI – Serial Peripheral Interface

SPI ist eine synchrone, serielle Punkt-zu-Punkt-Schnittstelle.

Signale:

- CS, SCLK, MOSI, MISO

Eigenschaften:

- Master-Slave
- Vollduplex

Übertragungsparameter:

- Clock Polarity (CPOL)
- Clock Phase (CPHA)

6.7 SPI-Hardwarestruktur

Ein SPI-System besteht aus:

- TX-Shiftregister
- RX-Shiftregister
- Clock-Divider
- FSM

Prüfungsregel: SPI ist vollständig synchron.

6.8 SPI-Beschreibung in VHDL

Die Beschreibung folgt einem festen Muster:

- Generics für Wortbreite und Mode
- FSM (Idle, Load, Shift, Done)
- SCLK aus Clock-Divider

6.9 I²C – Inter-Integrated Circuit

I²C ist eine synchrone Zweidraht-Bus-Schnittstelle.

Eigenschaften:

- SDA, SCL
- Multi-Master, Multi-Slave
- Open-Drain

6.10 I²C-Protokoll und Timing

Merkmale:

- Start: SDA fällt bei SCL = High
- Stop: SDA steigt bei SCL = High

alter
tschule

Serial Communication SPI: Hardware Implementation

General hardware structure

- Master / Slave concept
- TX / RX shift registers on both sides
- Controller on both sides
- 4 Lines to interconnect master with slave

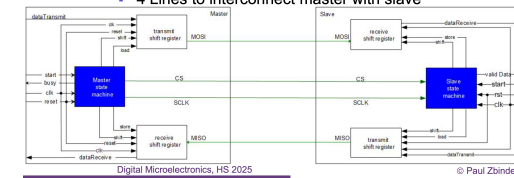


Abbildung 18: I²C Timingdiagramm mit Start/Stop/Acknowledge.

- Datenwechsel nur bei SCL = Low

Nach 8 Bit folgt ein Acknowledge-Zyklus.

6.11 I²C-Hardware und Arbitration

Durch Wired-AND-Struktur ist Arbitration möglich:

- Mehrere Master dürfen senden
- Master verliert, wenn er '1' sendet und '0' liest

Open-Drain in VHDL:

```
52 sda <= '0' when drive_low='1' else 'Z';
```

6.12 UART (Einordnung)

UART ist eine asynchrone Punkt-zu-Punkt-Schnittstelle.

Eigenschaften:

- Keine gemeinsame Clock
- Start- und Stop-Bits
- Einfache Hardware

Abschluss-Merkatz: UART ist einfach, aber unpräzise und langsam.

7 Parallel On-Chip Communication

7.1 Einordnung paralleler Kommunikation

Parallele Kommunikation überträgt mehrere Bits gleichzeitig über separate Leitungen. Sie wird primär **on-chip** eingesetzt, da physikalische Effekte off-chip die Skalierbarkeit stark begrenzen.

Merksatz (Prüfung): Parallel = hoher Durchsatz bei kurzer Distanz.

7.2 Serial vs. Parallel

Serielle Kommunikation überträgt ein Bit pro Leitung und benötigt Serialisierung und Deserialisierung. Parallele Kommunikation überträgt N Bits gleichzeitig.

- **Seriell:** wenige Leitungen, SERDES, höhere Latenz
- **Parallel:** viele Leitungen, einfache Logik, hoher Durchsatz

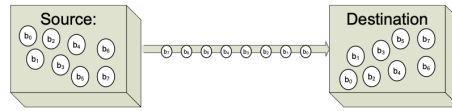
Prüfungsregel: Off-chip dominiert seriell, on-chip dominiert parallel.

7.3 Physikalische Grenzen paralleler Off-Chip-Kommunikation

Außerhalb des Chips begrenzen physikalische Effekte den Nutzen paralleler Übertragung:

- **Skew:** Laufzeitunterschiede zwischen Leitungen

One bit at a time



Multiple bits at a time

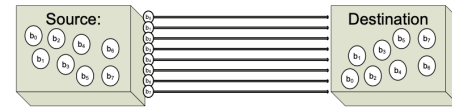


Abbildung 19: Serial vs. Parallel Kommunikation auf Bit-Ebene.

- **Crosstalk:** Kopplung benachbarter Leitungen
- Hoher Pad-Bedarf
- Hohe Kosten

Prüfungsregel: Mehr Leitungen → mehr Off-Chip-Probleme.

7.4 Vorteile paralleler On-Chip-Kommunikation

Auf dem Chip entfallen die meisten Nachteile:

- Keine Pads
- Kurze, kontrollierte Leitungen
- Geringer Skew
- Keine SERDES-Strukturen

Merksatz: On-Chip-Interconnects sind natürlich parallel.

7.5 Grundstruktur paralleler Schnittstellen

Eine parallele Schnittstelle besteht aus:

- Datenbus
- Steuerleitungen (Read, Write, Valid, Ready)
- Gemeinsamer Clock

Prüfungsregel: Parallele Interfaces sind immer synchron.

7.6 AXI als Standard-On-Chip-Interconnect

AXI (Advanced eXtensible Interface) ist Teil der AMBA-Architektur von ARM und de-facto Standard für SoC-Designs.

- Master-Slave
- Memory-Mapped
- Hohe Parallelität

Merksatz (Prüfung): AXI ist kanalbasiert, kein klassischer Bus.

7.7 AXI-Architektur

AXI verwendet fünf unabhängige Kanäle:

- Write Address (AW)
- Write Data (W)
- Write Response (B)
- Read Address (AR)
- Read Data (R)

Prüfungsregel: Read und Write sind vollständig getrennt.

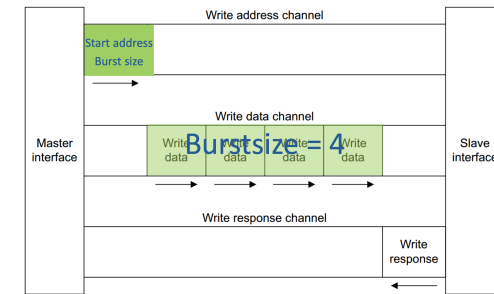


Abbildung 20: AXI Fünf-Kanal-Architektur für Read/Write.

All channels work along the same scheme:

1. Source generates valid signal as soon as information (data/address) is available
2. Destination generates ready signal to indicate that information (data or address) can be accepted
3. Transfer of data starts at first active clock edge after ready and valid are both high.

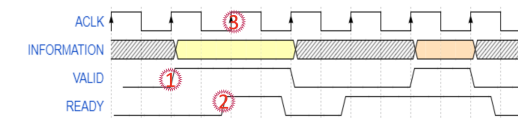


Abbildung 21: AXI Handshake Timingdiagramm (VALID/READY).

7.8 AXI-Handshake-Prinzip

Jeder Kanal nutzt dasselbe Handshake-Verfahren.

- Sender setzt VALID
- Empfänger setzt READY
- Transfer bei VALID & READY

Zentrale Regel (Prüfung): Daten müssen stabil bleiben, solange VALID=1 und READY=0.

7.9 AXI-Varianten

AXI existiert in mehreren Varianten:

- **AXI4:** Vollständig, Bursts bis 256 Transfers
- **AXI4-Lite:** Keine Bursts, Registerzugriffe
- **AXI4-Stream:** Datenstrom ohne Adressen

Prüfungsregel: Register-IP → AXI4-Lite.

7.10 AXI-Interconnects

Unterstützte Topologien:

- 1-to-1
- N-to-1 (Arbitration)
- 1-to-M (Address-Decoding)
- N-to-M

7.11 AXI auf Zynq

Der Zynq-SoC bietet mehrere AXI-Schnittstellen zwischen Processing System (PS) und Programmable Logic (PL):

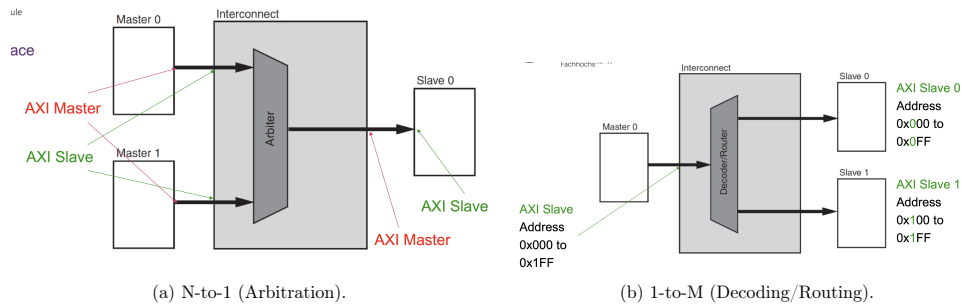


Abbildung 22: AXI Interconnect Topologien: N-to-1 und 1-to-M.

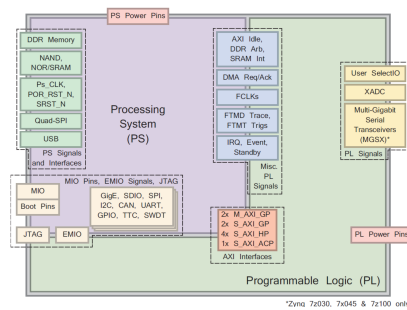


Abbildung 23: Zynq PS-PL AXI-Übersicht.

- General Purpose (GP)
- High Performance (HP)
- Accelerator Coherency Port (ACP)

7.12 AXI3 vs. AXI4

Im Zynq:

- PS nutzt AXI3
- Xilinx-IP nutzt AXI4

Prüfungsregel: Protokollkonverter lösen Inkompatibilitäten.

7.13 AXI-IP-Erstellung

Gängige Vorgehensweisen:

- Separater AXI-Interface-Block + User-Logik
- AXI-Peripheral-Template

7.14 Register-basierte Kommunikation

Kommunikation erfolgt über Memory-Mapped Register:

- CPU schreibt Register
- Logik liest Register
- Logik schreibt Status
- CPU liest Status

7.15 FPGA-Processor-Kommunikation

Integration erfolgt über:

- Hardware-Export
- BSP-Generierung
- Automatische Address-Header

Abschluss-Merkatz: AXI verbindet Software-Welt und FPGA-Logik deterministisch.

8 Debugging and Verification of SoC

8.1 Einordnung der Verifikation

Design Verification ist ein zentraler Bestandteil moderner SoC-Entwicklung und beansprucht häufig über 50 % der Entwicklungszeit.

Prüfungsmerksatz: Verifikation prüft richtiges Verhalten, nicht Performance oder Timing.

In dieser Vorlesung wird ausschließlich funktionale Verifikation betrachtet.

8.2 Funktionale Verifikation

Funktionales Verhalten lässt sich beschreiben als:

$$y = f(x, \text{state})$$

Prüfungsregel: Verifikation prüft, ob für gegebene Eingänge und Zustände die korrekten Ausgänge entstehen.

8.3 Exhaustive Verification

Exhaustive Verification testet alle möglichen Eingangskombinationen.

Prüfungsformel

Für n unabhängige Eingänge gilt:

$$N = 2^n$$

Prüfungsregel: Exhaustive Verification ist theoretisch korrekt, praktisch unmöglich.

8.4 Pragmatische Verifikationsstrategie

Da Exhaustive Verification nicht realisierbar ist, wird ein Kompromiss gewählt:

- Assertion-Based Verification
- Gezielte Testfälle
- Zufällige Stimuli
- Trennung von Design und Test

Merksatz: Ziel ist hohe Fehlerabdeckung, nicht Vollständigkeit.

8.5 Assertion-Based Verification

Assertions sind eingebaute Konsistenzprüfungen.

Typische Assertions

- Illegale FSM-Zustände
- Ungültige Adressen
- Über- und Unterläufe

Assertion – Code-Beispiel

```
53 assert addr < MAX_ADDR
54 report "Address out of range"
55 severity error;
```

Prüfungsregel: Assertions sind nicht für die Synthese bestimmt.

8.6 Testfälle und Stimuli

Effektive Testfälle decken unterschiedliche Szenarien ab:

- Normalbetrieb
- Grenzfälle
- Fehlerfälle
- Zufällige Daten

Prüfungsregel: Grenzwerte finden mehr Fehler als Mittelwerte.

8.7 Golden Model

Ein Golden Model liefert Referenzergebnisse für gegebene Stimuli.

Prüfungsregel

- DUT erzeugt Ergebnis
- Golden Model erzeugt Erwartungswert
- Automatischer Vergleich

Ohne Golden Model keine skalierbare Verifikation.

8.8 Test-Bench-Organisation

Strukturierte Test-Benches folgen festen Regeln:

- Periodische Clock
- Genau ein Stimulus pro Takt
- Antwortauswertung synchron zur Clock

Clock – Minimal-Code

```
56 clk <= not clk after 10 ns;
```

Prüfungsregel: Un synchron isierte Test-Benches erzeugen Scheinergebnisse.

8.9 Automatische Auswertung

Visuelle Inspektion ist unzureichend.

Prüfungsrezept

1. Stimulus erzeugen
2. DUT simulieren
3. Ergebnis speichern
4. Mit Golden Model vergleichen

Merksatz: Was nicht automatisch prüft, skaliert nicht.

8.10 File-basierte Test-Benches

VHDL unterstützt Datei-I/O über STD.TEXTIO.

Vorteile

- Große Testmengen
- Trennung von Daten und Code
- Wiederverwendbarkeit

Prüfungsregel: File-I/O ist Testbench-only.

8.11 Simulation und Modelle

Funktionale Simulation:

- VHDL oder Verilog

Timing-Simulation:

- Nur Verilog

Prüfungsregel: VHDL besitzt keine simprim-Timing bibliothek.

8.12 Debugging in Simulation

Simulation erlaubt vollständige Sichtbarkeit.

Werkzeuge

- Signal-Inspection
- Breakpoints
- Step-by-Step
- Signal-Forcing

Prüfungsregel: Simulation ist das mächtigste Debugging-Werkzeug.

8.13 Voraussetzungen zum Finden eines Fehlers

Ein Fehler wird nur gefunden, wenn alle drei Bedingungen erfüllt sind:

1. **Sensitization:** Fehler wird aktiviert
2. **Propagation:** Fehler breitet sich aus
3. **Observation:** Fehler ist sichtbar

Prüfungsmerksatz

Kein Output-Unterschied $\Rightarrow$ kein gefundener Fehler.

8.14 Stuck-at-Fehlermodell

Ein Signal ist permanent auf 0 oder 1 festgesetzt.

Prüfungsrezept

- Eingang so wählen, dass Fehler wirksam wird
- Pfad bis Ausgang offen halten
- Ausgang beobachten

Prüfungsregel: Stuck-at ist ein logisches, kein physikalisches Modell.

8.15 Debugging in Emulation

Auf Hardware sind interne Signale nicht direkt sichtbar.

Xilinx-Werkzeuge

- **VIO:** Setzen und Lesen interner Signale
- **ILA:** Logikanalysator im FPGA

Prüfungsregel: ILA ersetzt den Simulator auf Hardware.

8.16 Hardware/Software Co-Debugging

In SoCs ist gekoppeltes Debugging möglich.

Mechanismus

- Software-Breakpoint triggert ILA
- ILA-Trigger stoppt CPU

Abschluss-Merksatz: Gute Verifikation findet Fehler früh, billig und reproduzierbar.

9 Exam Code Templates

9.1 XDC: Physical + Timing (Skeleton)

```

1 ## --- Physical constraints ---
2 set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
3 set_property PACKAGE_PIN W17 [get_ports {clk}]
4
5 set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
6 set_property PULLUP true [get_ports {rst}]
7 set_property PACKAGE_PIN W20 [get_ports {rst}]
8
9 set_property IOSTANDARD LVCMOS18 [get_ports {x[*]}]
10 set_property PACKAGE_PIN T16 [get_ports {x[2]}]
11 # ... weitere Pins
12
13 set_property IOSTANDARD LVCMOS18 [get_ports {sda scl}]
14 set_property PULLUP true [get_ports {sda}]
15 set_property PULLUP true [get_ports {scl}]
16 # ... Pins for sda/scl
17
18 ## --- Clock constraint ---
19 create_clock -name clk -period 5.000 -waveform {0.000 2.000} [get_ports {clk}]
20
21 ## --- I/O delays (min/max) ---
22 set_input_delay -clock [get_clocks clk] -min -add_delay 1.000 [get_ports {x[*]}]
23 set_input_delay -clock [get_clocks clk] -max -add_delay 2.500 [get_ports {x[*]}]
24
25 set_output_delay -clock [get_clocks clk] -min -add_delay 0.500 [get_ports {y[*]}]
26 set_output_delay -clock [get_clocks clk] -max -add_delay 2.000 [get_ports {y[*]}]

```

9.2 VHDL: Simple Dual Port RAM (Block RAM inference)

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity sdp_ram is
6     generic(
7         ADDR_WIDTH : positive := 16;
8         DATA_WIDTH : positive := 8
9     );
10    port(
11        clk_wr : in std_ulogic;
12        clk_rd : in std_ulogic;
13        we : in std_ulogic;
14        addr_wr : in std_ulogic_vector(ADDR_WIDTH-1 downto 0);
15        addr_rd : in std_ulogic_vector(ADDR_WIDTH-1 downto 0);
16        din : in std_ulogic_vector(DATA_WIDTH-1 downto 0);
17        dout : out std_ulogic_vector(DATA_WIDTH-1 downto 0)
18    );
19 end entity;
20
21 architecture rtl of sdp_ram is
22     type mem_t is array (0 to 2**ADDR_WIDTH - 1) of std_ulogic_vector(DATA_WIDTH-1 downto 0);
23     shared variable mem : mem_t;
24
25     attribute ram_style : string;
26     attribute ram_style of mem : variable is "block";
27 begin
28     -- Write port (sync)
29     p_wr : process(clk_wr)
30     begin
31         if rising_edge(clk_wr) then
32             if we = '1' then
33                 mem(to_integer(unsigned(addr_wr))) := din;
34             end if;
35         end if;
36     end process;
37
38     -- Read port (sync)
39     p_rd : process(clk_rd)
40     begin
41         if rising_edge(clk_rd) then
42             dout <= mem(to_integer(unsigned(addr_rd)));
43         end if;
44     end process;

```

```

45 end architecture;

```

9.3 VHDL: I2C Open-Drain (one wire)

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity i2c_line is
5     port(
6         sda : inout std_logic;
7         drive_lo : in std_logic; -- '1' => pull low
8         sda_in : out std_logic
9     );
10 end entity;
11
12 architecture rtl of i2c_line is
13 begin
14     -- Open-drain: never drive '1'
15     sda <= '0' when drive_lo = '1' else 'Z';
16     sda_in <= sda;
17 end architecture;

```

9.4 VHDL: AXI4-Lite Registerbank (minimal)

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity axi_lite_regs is
6     generic(ADDR_W : positive := 6); -- byte address width
7     port(
8         aclk : in std_ulogic;
9         aresetn : in std_ulogic;
10
11         -- Write address
12         awvalid : in std_ulogic;
13         awready : out std_ulogic;
14         awaddr : in std_ulogic_vector(ADDR_W-1 downto 0);
15
16         -- Write data
17         wvalid : in std_ulogic;
18         wready : out std_ulogic;
19         wdata : in std_ulogic_vector(31 downto 0);
20
21         -- Write response
22         bvalid : out std_ulogic;
23         bready : in std_ulogic;
24
25         -- Read address
26         arvalid : in std_ulogic;
27         arready : out std_ulogic;
28         araddr : in std_ulogic_vector(ADDR_W-1 downto 0);
29
30         -- Read data
31         rvalid : out std_ulogic;
32         rready : in std_ulogic;
33         rdata : out std_ulogic_vector(31 downto 0)
34     );
35 end entity;
36
37 architecture rtl of axi_lite_regs is
38     type regfile_t is array (0 to 7) of std_ulogic_vector(31 downto 0);
39     signal regs : regfile_t := (others => (others => '0'));
40
41     signal aw_hs, w_hs, ar_hs : std_ulogic;
42     signal wr_addr_idx, rd_addr_idx : integer range 0 to 7;
43 begin
44     wr_addr_idx <= to_integer(unsigned(awaddr(5 downto 2))); -- word aligned
45     rd_addr_idx <= to_integer(unsigned(araddr(5 downto 2)));
46
47     awready <= '1';

```

```

48 wready <= '1';
49 arready <= '1';
50
51 aw_hs <= awvalid and awready;
52 w_hs <= wvalid and wready;
53 ar_hs <= arvalid and arready;
54
55 p_axi : process(aclk)
56 begin
57     if rising_edge(aclk) then
58         if aresetn = '0' then
59             regs <= (others => (others => '0'));
60             bvalid <= '0';
61             rvalid <= '0';
62             rdata <= (others => '0');
63         else
64             -- default: clear responses when accepted
65             if bvalid = '1' and bready = '1' then bvalid <= '0'; end if;
66             if rvalid = '1' and rready = '1' then rvalid <= '0'; end if;
67
68             -- write: accept address+data in same cycle (minimal model)
69             if (aw_hs = '1') and (w_hs = '1') then
70                 regs(wr_addr_idx) <= wdata;
71                 bvalid <= '1';
72             end if;
73
74             -- read: respond on address handshake
75             if ar_hs = '1' then
76                 rdata <= regs(rd_addr_idx);
77                 rvalid <= '1';
78             end if;
79         end if;
80     end if;
81 end process;
82 end architecture;

```

9.5 VHDL: Fixed Point ohne fixed_pkg (Template)

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  -- Beispiel: A,B sQ1.7 (8 Bit), Add => sQ2.7 (9 Bit), Mult mit C sQ1.7 => sQ3.14 (16 Bit)
6  architecture rtl of fp_no_pkg is
7      signal a1, b1, c1 : signed(7 downto 0); -- sQ1.7
8      signal x1          : signed(8 downto 0); -- sQ2.7 (guard bit)
9      signal y1          : signed(15 downto 0); -- sQ3.14
10  begin
11      -- Inputs: bereits skaliert angenommen
12      -- Guard Bit + korrektes Sign-Extend:
13      x1 <= (a1(a1'left) & a1) + (b1(b1'left) & b1);
14
15      y1 <= x1 * c1; -- Bitwachstum beachten
16
17      -- Output: passende Bits auswahlen (z.B. wieder 8 Bit sQ1.7)
18      -- hier: MSBs nehmen, LSBs abschneiden (Trunkierung)
19      Y <= std_logic_vector(y1(14 downto 7)); -- Beispiel-Slice, abhaengig vom Ziel-Q-Format
20  end architecture;

```

9.6 VHDL: Package + Function + Nutzung (Template)

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  package util_pkg is
6      subtype u16 is natural range 0 to 65535;
7      type triple_t is array (0 to 2) of u16; -- (0)=A, (1)=B, (2)=C
8      function pick0(x : triple_t) return u16;
9  end package;

```

```

11 package body util_pkg is
12     function pick0(x : triple_t) return u16 is
13     begin
14         return x(0);
15     end function;
16 end package body;
17
18 -- Usage
19 library ieee;
20 use ieee.std_logic_1164.all;
21 use ieee.numeric_std.all;
22 use work.util_pkg.all;
23
24 entity util_use is
25     port(
26         clk : in  std_ulogic;
27         rst : in  std_ulogic;
28         bus : in  std_ulogic_vector(47 downto 0);
29         out0 : out u16
30     );
31 end entity;
32
33 architecture rtl of util_use is
34     alias A_val : std_ulogic_vector(15 downto 0) is bus(47 downto 32);
35     alias B_val : std_ulogic_vector(15 downto 0) is bus(31 downto 16);
36     alias C_val : std_ulogic_vector(15 downto 0) is bus(15 downto 0);
37
38     signal t : triple_t;
39 begin
40     t <= (natural(to_integer(unsigned(A_val))),
41          natural(to_integer(unsigned(B_val))),
42          natural(to_integer(unsigned(C_val))));
43
44     p : process(clk)
45     begin
46         if rising_edge(clk) then
47             if rst = '1' then
48                 out0 <= 0;
49             else
50                 out0 <= pick0(t);
51             end if;
52         end if;
53     end process;
54 end architecture;

```